

```

CreateFileStream("netanim1.tr");
Simulator::Run();
cout << "\n\nAnimation XML file "<< animFile.c_str()<< "
created\n\n";
Simulator::Destroy();
return 0;
}

```

Even though we are concerned with animation and trace analysis in this article, I will briefly explain the program, in general. The script uses yet another helper class called *PointToPointDumbbellHelper* to generate the topology. This class will help us create dumbbell shaped topologies. But the line '*PointToPointDumbbellHelper d(1, pointToPointEndNode, 1, pointToPointEndNode, pointToPointRouter);*' creates a topology with just one end node (leaf node) on either side of the handle of the dumbbell formed by two nodes. All the other lines of code in the script are somewhat similar to the ones we have seen in the previous ns-3 script, except for those lines that are responsible for generating the XML based animation trace file for NetAnim. In the next section, we will discuss these lines.

NetAnim execution

In the simulation script *netanim1.cc* the line '*std::string animFile = "netanim1.xml";*' creates an animation trace file with the name *netanim1.xml*. After generating the topology, the line '*d.BoundingBox(1, 1, 25, 25);*' defines the Cartesian coordinates of the animation window. The line '*AnimationInterface anim(animFile);*' sends the data to the file '*netanim.xml*' required by NetAnim for displaying the topology animation. Finally, the line '*anim.EnablePacketMetadata();*' adds packet metadata related information to the animation trace file. This line of code is optional. When you execute the ns-3 script, the file *netanim1.xml* is generated in the directory *ns/ns-*

allinone-3.22/ns-3.22. Now open this file in the NetAnim window by clicking the *file* menu, and after opening the file, start the topology animation by pressing the *Play* button. Figure 1 shows the network traffic in the NetAnim window.

If you observe the NetAnim window carefully, you will see different icons denoting operations like start, stop, reload, display packet metadata, zoom in, zoom out, etc. Play around with those buttons for some time till you feel confident about working with NetAnim. The top right corner displays the current simulation time. The speed of the animation can be increased or decreased by moving a button to the left or right. This button is placed at the top panel between the words 'fast' and 'slow'. If you observe Figure 1, you will see that I have reduced the speed of the animation to the minimum possible value to take the screenshot. Now that we have some knowledge of NetAnim, it is time to deal with the ASCII trace files.

ASCII trace files demystified

The three important aspects of ns-3 are simulating the correct topology, understanding the structure of an ASCII trace file, and making changes to the existing ns-3 source files to suit your proposed protocol or architecture. In ns-3, almost all the information is available in the form of an ASCII trace file. Getting results from an ns-3 simulation involves deciphering the trace file. The simulation script *netanim1.cc* also produces an ASCII trace file named *netanim1.tr*. The lines of code '*AsciiTraceHelper ascii;*' and '*pointToPointEndNode.EnableAsciiAll(ascii.CreateFileStream("netanim1.tr"));*' are responsible for generating the trace file. The trace file *netanim1.tr* is a large file with a number of lines of data. Each line in the trace file corresponds to a trace event. Trace events denote what happens to a packet in the transmit queue of a node. The important events happening in the transmit queue include packet enqueueing denoted by '+', packet dequeuing denoted by '-', packet reception denoted by 'r' and packet drop denoted by 'd'. Those who are familiar with ns-2 might remember that these symbols carry the same meaning in ns-2 also. Now consider the code shown below. It is actually a single line of data from the trace file *netanim1.tr* divided into a number of lines for better understanding.

```

+
2.03474
/NodeList/1/DeviceList/8/$ns3::PointToPointNetDevice/TxQueue/
Enqueue
ns3::PppHeader (
Point-to-Point Protocol: IP (0x0021))
ns3::Ipv4Header (
tos 0x0 DSCP Default ECN Not-ECT ttl 63 id 0 protocol 17
offset (bytes) 0 flags [none]
length: 1052 192.169.100.1 > 192.168.100.1)
ns3::UdpHeader (

```

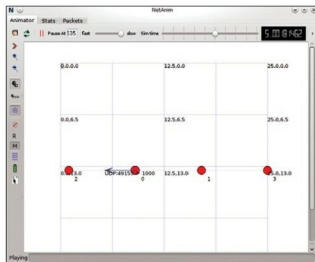


Figure 1: Network traffic in NetAnim